

forEachLoopDemo3.java

```
//for-each loop
void main()
{
    char []characters = {'a','b', 'c', 'd', 'e'};

    for(char character : characters)
    {
        IO.println(character);
    }
}
```

Can we hold different types of values using array ?

In java, there is a predefined class called **Object available in java.lang package**. By using this Object class we can **hold all different types of values in java**.

```
//for-each loop
void main()
{
    Object []values = {12, 31.90, 'A', "Java", true };

    for(Object value : values)
    {
        IO.println(value);
    }
}
```

We are able to hold all different types of values because all data are of type Object

How to sort the data ?

\* We have a predefined class called **Arrays** available in java.util package.

\* It is a utility class [helper class which contains only static method] which provides various static methods :

a) public static void sort(int []arr) : Used to sort integer array data in ascending order.

b) public static void sort(Object [] arr) : Used to sort custom data like String , Wrapper class data.

Wap to sort integer data in ascending order :

```
void main()
{
    int []jarr = {50, 30, 10,20, 90};

    IO.println("Original Data :");

    for(int val : arr)
    {
        IO.println(val);
    }

    Arrays.sort(arr); //sort the array in ascending order

    IO.println("After sorting");

    for(int val : arr)
    {
        IO.println(val);
    }
}
```

WAP to sort the city name in alphabetical order :

```
void main()
{
    String []cities = {"Hyd", "Pune", "Ameerpet", "Bbsr", "Delhi", "Vizag"};

    Arrays.sort(cities);

    for(String city : cities)
    {
        IO.println(city);
    }
}
```

In compact source file approach we need not write import statement for importing a class, In this approach java compiler will automatically import all the classes which are available in **java.base module [java 9v -> JPMS (Java platform module system)]**

All possible combinations of methods :

The following cases are describing all the possible combinations of methods.

Case 1:

Method with no return type and no parameter.

Example :

```
public void accept()
{
}
```

Case 2:

Method with return type and no parameter.

Example :

```
public int accept()
{
    return 0;
}
```

Case 3:

Method with no return type and with parameter.

Example :

```
public void accept(int x, int y)
{
}
```

Case 4:

Method with return type and with parameter.

Example :

```
public int accept(int x, int y)
{
    return x+y;
}
```

Calling a method using IO.println() OR System.out.println() :

\* We can directly call the methods from IO.println() OR System.out.println() statements, the only limitation is , We cannot call a method whose return type is **void**.

Case 1:

```
void main()
{
    IO.println(m1()); //Error [Invalid Statement] -> m1() method return type is void so we cannot call from IO.println()
}
```

void m1()

```
{
}
```

Case 2 :

```
void main()
{
    IO.println(show());
    IO.println(display());
    IO.println(add(12, 8));
}
```

int show()

```
{
    return 9;
}
```

double display()

```
{
    return 15.29;
}
```

int add(int x, int y)

```
{
    return x + y;
}
```

Numerical Logical Programs :

Points :

a) We should use while loop when the number iterations are not fixed, If fixed then use for loop.

b) void main()

```
{
    IO.println(10 % 3); //1
    IO.println(9 % 3); //0
    IO.println(2 % 3); //2
    IO.println(1 % 3); //1

    IO.println("=====");
    IO.println(15/10); //1
    IO.println(10/10); //1
    IO.println(1/2); //0
}
```

WAP to count the number of digits in a number :

Example :

```
1234 = 4
999  = 3
12   = 2
000  = 1
```

```
void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Total digits are :"+getDigitCount(number));
}
```

int getDigitCount(int num)

```
{
    int count = 0;

    if(num==0)
    {
        return 1;
    }

    while(num !=0)
    {
        num = num /10;
        count++;
    }

    return count;
}
```

WAP to find the sum of all the digits

```
45   = 9
123  = 6
89   = 17
```

```
void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Sum of all digits are :"+getSumOfDigits(number));
}
```

int getSumOfDigits(int num) //123 //1 + 2 + 3

```
{
    int sum = 0; //6

    while(num !=0) //num = 0
    {
        int digit = num % 10;
        sum = sum + digit;
        num = num /10;
    }

    return sum;
}
```